

[Dreamhack.io] pwn_patch_2 Write-Up

- Platform: Dreamhack
- Date: 2026-08-27 (solved) / 2026-08-28 (written)
- Difficulty: Medium
- Tags: `#pwn` `#patch` `#command_injection` `#shellcode`

1. Challenge Overview (문제 개요)

📄 문제 요약

- **목표:** 프로그램 내 존재하는 취약점 패치
- **제약 조건:** 최대 120 바이트 패치 가능, 원본 바이너리와 입출력 동일할 것

- **제공 파일:** `cmdi_origin` `.DS_Store`

- **보호 기법 (Checksec):**

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
undefined8 main(void)
{
    EVP_PKEY_CTX *ctx;
    long in_FS_OFFSET;
    undefined1 input_ip [32];
    EVP_PKEY_CTX command [136];
    long canary;

    canary = *(long *)(&in_FS_OFFSET + 0x28);
    memset(input_ip, 0, 0x20);
    ctx = command;
    memset(ctx, 0, 0x80);
    init(ctx);
    printf("IP: ");
    read(0, input_ip, 0x1f);
    snprintf((char *)command, 0x20, "/bin/ping -c 3 %s", input_ip);
    system((char *)command);
}
```

```

if (canary != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return 0;
}

```

- `input_ip` : 사용자의 입력값
- `command` : `system` 함수를 통해 실행되는 명령어 저장 변수

프로그램 흐름

- 사용자의 입력을 받고, `/bin/ping -c 3 %s` 포맷에서 `%s` 부분에 사용자 입력값을 넣어 명령어를 만든다.
- 만들어진 명령어를 `system` 변수로 넣어 실행한다.

system 함수와 execve 함수

- `execve()` :
 - 첫 번째 전달인자 `pathname` : 실행할 파일의 경로
 - 두 번째 전달인자 `argv[]` : 실행할 파일에 전달할 인자 배열
 - 세 번째 전달인자 `envp[]` : 실행할 파일에 전달할 환경변수 배열
 - `pathname` 경로에 있는 파일을 `argv[]` 인자와 `envp[]` 인자로 실행한다.
 - 타켓 프로그램이 직접 실행되는 것이므로, 셸을 파일 경로로 지정하지 않는 이상 **셸을 거치지 않는다.**
- `system()` :
 - 문자열을 전달인자로 받고, 내부적으로 `do_system()` 을 호출한다. 해당 함수는 `execve()` 함수를 통해 전달인자로 받은 문자열을 셸 명령어로서 실행한다.
 - `execve()` 함수를 호출할 때, `pathname` 이 `/bin/sh` 등의 셸이고, `argv[]` 은 `["/sh", "-c", "사용자 입력 문자열"]` 로 실행이 된다.
 - 따라서 해당 함수는 **셸을 열고 명령어를 실행**하기 때문에, 사용자 입력값에 대한 검증이 없다면 원하는 명령어를 이용해 셸을 획득할 수 있게 된다.

특징	<code>system("command")</code>	<code>execve(pathname, argv[], envp[])</code>
셸 호출 여부	호출함	호출 안 함 (커널이 직접 실행)
특수문자 처리	명령어 구분자로 해석됨	단순 문자열 데이터로 취급됨

취약점 원인 (Root Cause)

- 발생 원인: 입력값 검증이 없으므로 `command injection` 가능
- 파급 효과: 셸 호출 가능

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

`system` 함수를 호출하는 명령어를 `jmp` 명령을 통해 다른 영역으로 이동한 뒤, `execve("/bin/ping", ["/bin/ping", "-c", "3", "사용자 입력값"], NULL)` 형식에 맞게 레지스터를 설정하고 호출하는 코드로 패치한다.

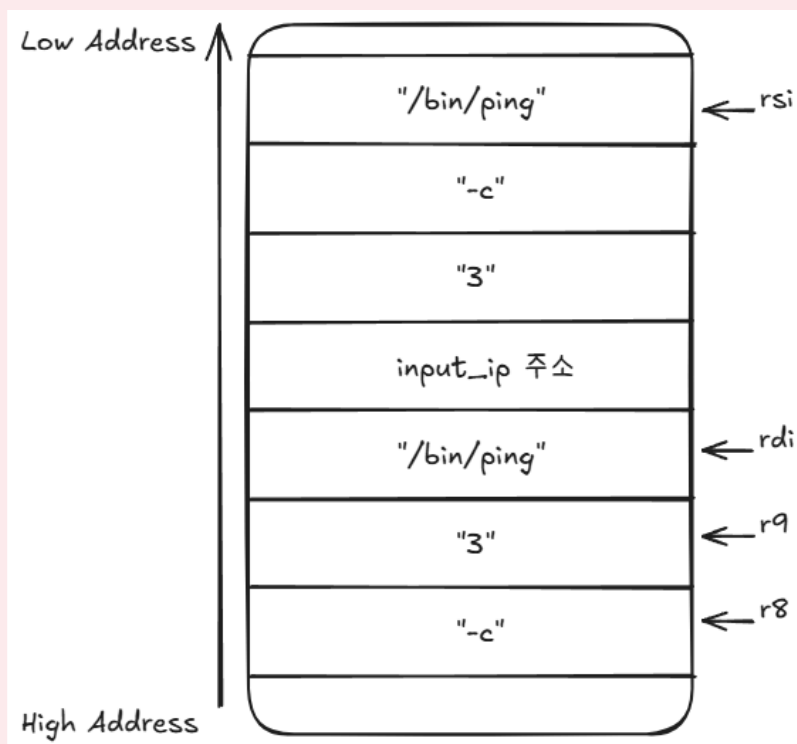
1. `system` 콜이 아닌 `execve` 콜로 바꾼다.
2. 64비트 환경 SYSV 호출 규약을 맞추기 위해 `rdi`, `rsi`, `rdx`를 각각 `"/bin/ping"`, `["/bin/ping", "-c", "3", "사용자 입력값"]`, `NULL`로 맞춰줘야 한다.
3. 원본 바이너리에서 `system()` 콜 이후 수행하는 작업이 없으므로 (단순 canary 검사 및 종료) `execve` 콜을 다이렉트로 실행해서 바이너리 메모리를 통째로 `execve`로 바꾼다.

⚠ 만약 원본 바이너리에서 `system` 콜 이후 추가로 수행하는 작업이 있다면 `execve` 콜 이후 메인 함수로 돌아오기 위해, `execve` 콜 이전에 `fork` 함수를 통해 자식 프로세스를 생성하고, 자식 프로세스에서 `execve` 콜을 하는 과정이 추가로 필요할 것이다.

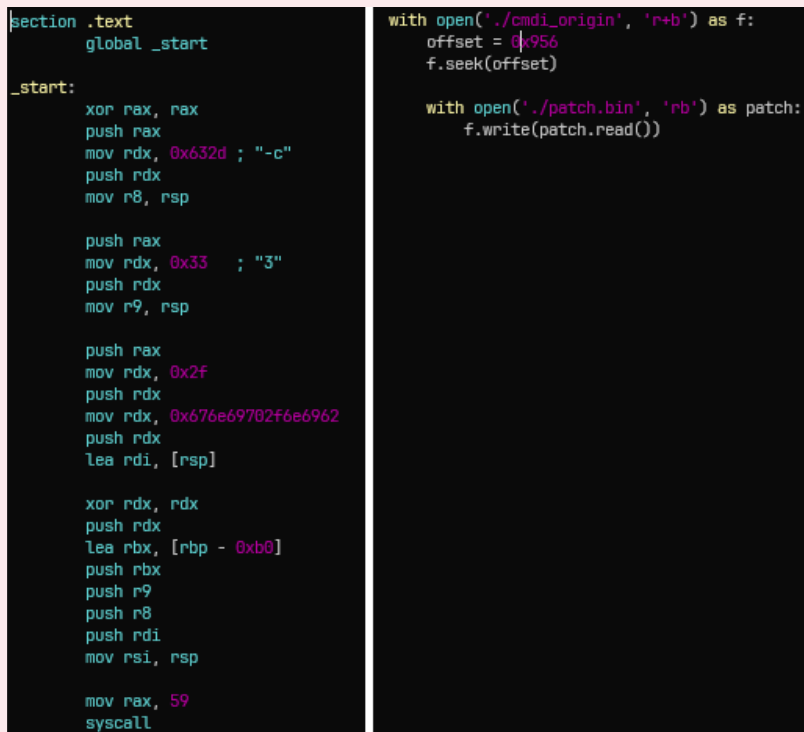
3. Trial & Error (삽질 및 실패 기록)

× Attempt 1: [`main()` 에서 그냥 바꾸면 되겠지?]

- 가설: ghidra의 Assemble 기능을 통해 `main()`의 코드를 바꾸면 될 것이다.
- 시도 내용:
 - 바꾸고자 하는 어셈블리 코드를 만든 뒤, 해당 바이너리를 파이썬을 통해 주입
 - 어셈블리어는 인자값들을 다음과 같이 스택에 저장하고, 레지스터가 가리킬 수 있도록 하는 코드이다.



- 아래 사진 왼쪽이 어셈블리 코드 전체(.asm 파일), 오른쪽 사진은 nasm, objcopy 명령을 통해 바이너리 코드로 만든 파일을 원본 바이너리에 패치하는 파이썬 코드이다.



```

section .text
global _start

_start:
    xor rax, rax
    push rax
    mov rdx, 0x632d ; "-c"
    push rdx
    mov r8, rsp

    push rax
    mov rdx, 0x33 ; "3"
    push rdx
    mov r9, rsp

    push rax
    mov rdx, 0x2f
    push rdx
    mov rdx, 0x676e69702f6e6962
    push rdx
    lea rdi, [rsp]

    xor rdx, rdx
    push rdx
    lea rbx, [rbp - 0xb0]
    push rbx
    push r9
    push r8
    push rdi
    mov rsi, rsp

    mov rax, 59
    syscall
  
```

```

with open('./cmdi_origin', 'r+b') as f:
    offset = 0x956
    f.seek(offset)

    with open('./patch.bin', 'rb') as patch:
        f.write(patch.read())
  
```

- 결과 및 에러: Segmentation Fault 발생
- 원인 분석:
 - 패치하고자 하는 위치가 정확히 맞지 않다는 점과 패치하고자 하는 바이너리의 전체 크기를 계산하지 않은 채 주입했기 때문에 원본 바이너리의 일부 명령어들을 패치할 바이너리로 덮어씌워졌고, 그로 인해 Opcode가 깨지면서 Segmentation Fault 가 발생하였다.

- 이 과정 이후, 더 이상 패치 코드를 주입할 방법이 생각 나지 않아 풀이를 참고하였다.
- 풀이를 통해 알게 된 내용은 다음과 같다.

💡 ELF 파일 내 메타데이터가 존재하는 실행 가능한 영역

- .eh_frame :
 - C++ 등에서 try-catch 구문의 예외 처리에 대해 되감기(Rewind)를 위해 메타데이터가 존재하는 영역
 - 해당 영역은 텍스트로만 나열되어 있는 것처럼 보이지만, 실제로 권한을 확인해보면 실행 권한이 존재한다는 것을 알 수 있다.

따라서 예외 없이 정상적으로 실행되는 프로그램에서는 이 영역을 한 번도 건드리지 않는다는 말이다. 우리는 이 영역에 바이너리 코드를 삽입함으로써 패치를 진행할 것이다.

이렇게 실행 권한이 존재해서 명령어를 삽입할 수 있는 공간을 Code Cave라고 부른다.

5. Exploit Code (최종 코드)

```
# ===== 풀이를 통해 확인한 패치 주입 코드 =====
#-*- coding:utf-8 -*-
from pwn import *
import base64
context.arch='x86_64'
e = ELF('./cmdi_origin_bck')
eh_frame = e.get_section_by_name('.eh_frame').header.sh_addr
target_address = e.address+0x98A # call system()
def patch_execve():
    data = '''
    lea    rax,[rip+ping]
    mov    QWORD PTR [rbp-0xe0],rax
    lea    rax,[rip+opt]
    mov    QWORD PTR [rbp-0xd8],rax
    lea    rax,[rip+cnt]
    mov    QWORD PTR [rbp-0xd0],rax
    lea    rax,[rbp-0xb0]
    mov    QWORD PTR [rbp-0xc8],rax
    mov    QWORD PTR [rbp-0xc0],0x0
    mov    rdx, 0
    lea    rax,[rbp-0xe0]
    mov    rsi,rax
    lea    rdi,[rip+ping]
    mov    rax, 59
    syscall
    ping: .asciz "/bin/ping"
    opt: .asciz "-c"
    cnt: .asciz "3"
    '''
    print(len(asm(data)))
    e.write(eh_frame, asm(data)) # 1
    print('eh_frame : '+hex(eh_frame))
    e.write(target_address, asm('call
{}'.format(hex(eh_frame)),vma=target_address)) # 2
    e.save("./cmdi_patched")
def send_binary():
    s = remote('host3.dreamhack.games', 20377)
    with open('./cmdi_patched', 'rb') as f:
        buf = base64.b64encode(f.read())
    s.sendline(buf)
    s.interactive()
patch_execve()
send_binary()
```

- 이 코드를 실행했을 때 나오는 결과는 아래와 같다.

```
115
eh_frame : 0x400aa0
[+] Opening connection to host3.dreamhack.games on port 13426: Done
[*] Switching to interactive mode
Give me patch binary (base64): \x7fELF..
[+] Correct: 107
[+] Success to function check
[+] Success to cmdi check
DH{_____}
```

- 파이썬 `pwntools` 라이브러리를 통해 어셈블리어를 `.eh_frame` 영역 시작 주소에 주입한다.
- `main` 함수에서 `system()` 호출이 되는 명령어를 `call [.eh_frame 시작 주소]` 명령어도 바꾼다. (이때 `Offset`만 바뀌므로, 뒤의 코드들이 변조될 일이 없다.)
 - `call`, `jmp` 등 `rip`의 값이 바뀌면서 실행 흐름을 바꾸는 명령은 인자로 주소가 아닌 `offset`을 전달한다. (이 방법을 상대 주소 지정 방식이라고 부른다.)
 - `offset`을 구하는 방법은 목적지 주소에서 현재 `call`, `jmp` 명령이 있는 주소 + 5 를 빼면 구할 수 있다.

① 상대 주소 지정 방식에서 Offset을 구하는 원리 >

- `call`, `jmp` 등의 명령어는 뒤에 있는 인자값을 `rip`에 더함으로써 다음에 실행될 명령어 주소로 이동한다.
- 이때, **더해지는 시점**이 `call`, `jmp` 등의 명령어 **다음**에 더해진다.
- `call`, `jmp` 등의 명령어는 인자로 4바이트의 `offset`을 받고, 명령어 자체의 `Opcode`는 1바이트이므로, 총 5바이트가 필요하다.
- 따라서 더해지는 시점의 주소는 `call, jmp 명령어 주소 + 5`가 된다.
- 구하는 `offset`은 `offset = (목적지 주소) - (call, jmp 명령어 주소 + 5)`가 되고, `call offset`, `jmp offset` 등을 통해 실행 흐름을 바꿀 수 있다.

내가 만들었던 어셈블리 코드는 실행이 될까?

- 삽질을 통해 `Segmentation Fault`를 받은 나의 어셈블리 코드를 `.eh_frame` 영역에서 실행하면 될까 궁금해서 위의 풀이 코드 중 `data`의 값을 내 어셈블리 명령어로 바꿔서 실행해보았다.

```
data = '''
xor rax, rax
push rax
mov rdx, 0x632d
push rdx
mov r8, rsp

push rax
mov rdx, 0x33
push rdx
mov r9, rsp

push rax
mov rdx, 0x2f
push rdx
mov rdx, 0x676e69702f6e6962
push rdx
lea rdi, [rsp]

xor rdx, rdx
push rdx
lea rbx, [rbp - 0xb0]
push rbx
push r9
```

```

push r8
push rdi
mov rsi, rsp

mov rax, 59
syscall
'''

```

- 바꾼 부분은 삽질했을 때의 코드와 동일하고, 결과는 아래와 같다.

```

80
eh_frame : 0x400aa0
[+] Opening connection to host3.dreamhack.games on port 13426: Done
[*] Switching to interactive mode
Give me patch binary (base64): \x7fELF..
[+] Correct: 82
[+] Success to function check
[+] Success to cmdi check
DH{ }

```

- 풀이 코드랑 비교해보았을 때, 패치 바이너리 코드 바이트 수가 35바이트 차이가 났다.
- 이는 값들을 스택에 저장하는 방식의 차이에서 나오는데
 - 풀이의 경우 메모리에 저장된 문자열의 주소를 레지스터에 저장하는 과정과, 그 레지스터의 값을 스택에 push 하는 두 단계를 거친다.
 - 또한 문자열을 다른 메모리 위치에 저장하는 과정까지 포함되어 있어 바이트 수가 증가한 것이다.
 - 내가 만든 어셈블리의 경우, 레지스터에 64비트 즉시값을 대입하고, 그걸 push하는 명령을 통해 문자열을 스택에 저장한다.
 - 이는 다른 메모리 위치에 저장된 문자열을 가지고 오는 과정보다 opcode 길이가 짧기 때문에 최종적으로 35바이트 길이 차이가 난 것으로 보인다.

6. 배운 점 & 오답 노트

- 새로 알게 된 개념:
 - `.eh_frame` : 예외 처리를 위해 만든 공간이지만 실행 권한이 있는 영역
 - 상대 주소 지정 방식: `call`, `jmp` 등의 명령은 인자로 `offset`을 받아 `rip`에 더하는 방식으로 이루어진다. 이때 더해지는 시점이 그 다음 명령어 주소이므로 이를 주의해야 한다.
- 새로 알게 된 pwntools 라이브러리 함수:
 - `write(address, binary)` : `address`에 `binary`를 주입할 수 있음
 - `asm("call {}".format(hex(eh_frame)), vma="address")` : `call`, `jmp` 등 상대 주소 지정 방식의 명령어를 이용할 때, 해당 명령어가 가상 메모리 기준 `address`에 주입될 것임을 알려주어 `offset`을 알아서 계산해 넣어준다. 이를 통해 어셈블리 명령어에서 `offset`이 아닌 목적지 주소를 넣어도 알아서 계산해 넣어준다.
 - `get_section_by_name('섹션 이름')` : 실행 파일에 있는 섹션 이름에 해당하는 영역의 정보를 가지고 온다. (구조체 형태로 되어 있어서 이후 `.header.sh_addr` 등 원하는 정보만 가지고 올 수 있도록 조정할 필요가 있다.)